Sprint 3 Retrospektive

Projekt: KI-gestütztes System zur frühzeitigen Erkennung von Datenlecks

Sprint: Sprint 3

The primary goal of Sprint 3 was to make the system components developed in previous sprints more stable, reliable, and interoperable.In particular, efforts were focused on the robustness of the data collection process, company matching accuracy, LLM-powered analysis, dashboard–backend integration, and frontend usability. The goal was to transform the project from a mere working prototype into a more integrated and presentation-ready system.

Throughout the sprint, team members worked in parallel on different system components. On the Collector side, the plan was to update existing sources, add new onion sources, make Tor connections more stable, and improve the CAPTCHA-solving process. On the backend side, the goals were to complete the APIs required for the dashboard, develop source management functions, and make the descriptions generated by the LLM accessible via the frontend. On the analytics side, the goals were to reduce company-matching errors, decrease the number of findings classified as "Unknown," and ensure the LLM-supported explanation mechanism operates in a more controlled manner. On the frontend side, plans included correcting dashboard data, resolving visual issues, improving filtering and pagination behavior, enabling report export, and ensuring charts can display more comprehensive time ranges.

By the end of the sprint, significant improvements were made in various parts of the system. The company matching logic was reorganized, and the company catalog was expanded. As a result of these efforts, the number of "Unknown" findings which initially exceeded 15,000 was reduced to approximately 560. LLM analyses were converted to an on-demand structure that analysts can initiate via the "Analyze with AI" option, and the ability to save and reuse the generated results was enabled.On the Collector side, a Moondream-based fallback mechanism was added for cases where Ollama is unavailable. Timeout and retry behavior was developed to handle slow Tor nodes. Unreachable onion sources were reviewed, and the source list was updated.On the frontend, the "Findings" section was simplified, pagination was added to the "Source Management" area, and the dashboard was made more compact and readable. The time ranges displayed in the graphs were expanded. The previously problematic export function was improved, enabling reports to be both saved as PDFs and printed.

Problems Encountered and Measures Taken:

One of the most significant problems encountered during the sprint was related to data quality. Since short company aliases, such as "MS," matched even within unrelated words, incorrect company assignments were occurring. Additionally, some automatically extracted company names were too general, making it difficult to identify the correct company. This situation not only affected the company distribution on the dashboard but also reduced the reliability of the analysis results. To resolve the issue, safer word boundaries were used when checking short aliases, and the company catalog was expanded. Despite this, it was observed that some title-based company matches still required human review.

However, with LLM integration, since there were no previously recorded analysis results for many findings, only fallback explanations were displayed. In addition, the rate limits of the GitHub Models service were causing recurring 429 errors. Automatically analyzing all findings not only generated unnecessary requests but also led to the service being throttled in a short

period of time. Therefore, it was decided to initiate analyses only upon the analyst's request, rather than running them automatically. By saving the generated results, we prevented repeated requests for the same finding.

Ollama, used in the CAPTCHA solving process, would occasionally fail to respond due to high traffic or return a "server unreachable" error. This situation could cause the data collection process to halt for resources protected by CAPTCHA. To address this issue, the Vision-Language-based Moondream model was integrated as an alternative. The system initially uses Ollama, but automatically switches to the fallback model when Ollama fails. This approach aims to prevent temporary issues with the primary model from halting the entire data collection process.

It was observed that some nodes on the Tor side were responding very slowly, significantly prolonging the scanning time. For this reason, a five-second timeout mechanism was developed. If a node does not respond within the specified time, the connection is terminated and a retry is made via a different Tor node. Additionally, it was determined that some onion sites were no longer accessible. These resources were reviewed, and new active onion sites were added to the system.

On the frontend side, there were various issues related to layout, theme, pagination, and data display. While some areas appeared unnecessarily complex, certain functions were not working consistently across different sections of the dashboard. In particular, the findings table, filters, result counts, and pagination behavior were re-examined. Issues that occurred during PDF saving in the Export section were resolved.

Quality, Code, and Process Assessment:

Throughout the sprint, care was taken to record changes as small, clear commits and to include the relevant issue numbers in the commit messages. Before making extensive changes to the data, a dry-run was performed to verify potential outcomes. Risky operations, such as company matching and reprocessing "Unknown" findings, were carried out in the form of phased backfills. This prevented a potential error from affecting all data at once.Additional tests were performed on the backend, detector, and API sides. Frontend changes were validated using ESLint and a production build. Following a Docker import issue, verifying actual container startups and API endpoints was also added to the testing process.

Throughout the sprint, collaboration within the team proceeded smoothly and without conflict. Team members worked on their own issue branches, and the developed code was merged according to the established order. Since the entire team was aware of when each change would be merged, potential merge conflicts were largely prevented. Thanks to quick feedback and a clear commit history, the sources of any issues that arose were easier to identify.Occasional rework was necessary, particularly regarding data quality and Docker integration. We sought to strike a balance between covering more findings in company matching and reducing false positives. During this process, we used dry-runs and phased data updates rather than making broad changes directly. There were no significant personal conflicts within the team, and technical differences were resolved by evaluating test results.

Throughout the sprint, AI tools were used in areas such as code analysis, test generation, preparing backfill scripts, developing the company list, frontend adjustments, and documentation. These tools were particularly helpful in reviewing the existing code more

quickly and identifying potential solutions.However, it became clear that the results generated by AI must be verified. In some cases, incorrect assumptions were made about the structure of Docker modules, or overly generic company names were suggested. A feature created on the frontend might work correctly in one section but cause issues in another. The AI occasionally added unwanted features or altered existing behavior without realizing it.For this reason, AI outputs were not accepted outright. The changes made were reviewed and tested by team members and, when necessary, revised through new requests. The sprint experience demonstrated that AI is a useful tool for accelerating the development process but cannot replace human oversight, actual build testing, and manual verification.

Overall, Sprint 3 was a success. Not only were new features added during this sprint, but data quality, system stability, and frontend usability issues carried over from previous sprints were also largely addressed. In particular, reducing “Unknown” findings, improving company matching, implementing a CAPTCHA fallback mechanism, and enhancing the dashboard made the project more reliable and understandable.

The upcoming sprint will be the project’s final sprint. Therefore, the focus will be on resolving any remaining bugs, completing any outstanding tests, and fully finalizing the project to make it ready for presentation.